

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 13

Nama:
RAJATA PRINGGIT BINTANG BIHAR

NIM:
E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada aplikasi Acara 12, Controller masih menangani validasi data secara langsung sebelum data disimpan ke database melalui Repository. Semakin banyak aturan validasi yang dibutuhkan, seperti NIM wajib diisi, NIM tidak boleh duplikat, format email harus valid, dan program studi harus tersedia, Controller menjadi semakin panjang karena mencampur urusan request/response dengan logika bisnis.

Praktikum Acara 13 membahas penerapan Service Layer untuk memisahkan logika bisnis dari Controller, serta Flash Message menggunakan Session untuk menampilkan notifikasi satu kali (tampil sekali lalu dihapus) setelah proses tambah atau ubah data.

2. Tujuan

1. Membuat class MahasiswaService yang menerima MahasiswaRepository dan ProdiRepository melalui constructor.
2. Memindahkan seluruh proses validasi data mahasiswa dari Controller ke MahasiswaService.
3. Membuat method create() dan update() pada MahasiswaService untuk menangani proses tambah dan ubah data.
4. Memastikan MahasiswaController tidak lagi melakukan validasi maupun query database secara langsung.
5. Membuat mekanisme Flash Message menggunakan \$_SESSION yang menampilkan pesan satu kali kemudian menghapusnya.
6. Menguji seluruh proses (tambah, ubah, flash message) melalui browser.

3. Dasar Teori

Service Layer

Pada aplikasi sederhana, Controller mungkin masih bisa menangani validasi dan proses penyimpanan data. Namun, ketika aplikasi semakin besar, Controller akan menjadi terlalu panjang dan sulit dipelihara. Untuk mengatasi hal ini, digunakan Service Layer: Service bertugas menangani logika bisnis aplikasi (validasi, pengecekan data duplikat, aturan bisnis lain), sedangkan Controller hanya menerima request, memanggil Service, lalu menentukan response. Dengan pembagian tugas Controller → Service → Repository, Controller tetap tipis dan Service menjadi satu-satunya tempat untuk logika bisnis.

Flash Message dengan Session

Setelah data berhasil ditambahkan atau diubah, pengguna perlu mendapatkan informasi hasil prosesnya, misalnya “Data mahasiswa berhasil ditambahkan.”. Pesan seperti ini disimpan sementara pada Session dan disebut Flash Message, yaitu pesan yang hanya ditampilkan satu kali kemudian dihapus dari Session agar tidak muncul lagi ketika halaman di-refresh.

HASIL PRAKTIK

2. Implementasi

a. MahasiswaService (Service Layer)

MahasiswaService (app/Services/MahasiswaService.php) menerima MahasiswaRepository dan ProdiRepository melalui constructor, lalu menyediakan method create() dan update() yang di dalamnya memanggil validate() sebelum data diteruskan ke Repository.

```
class MahasiswaService
{
    public function __construct(
        private MahasiswaRepository $repo,
        private ProdiRepository $prodiRepo
    ) {}

    public function create(array $input): array
    {
        $data = $this->normalize($input);
        $errors = $this->validate($data);

        if (!empty($errors)) {
            return $this->failed($errors, $data);
        }

        try {
            $id = $this->repo->create($data);
        } catch (PDOException $e) {
            return $this->databaseFailed($e, $data);
        }

        return [
            'success' => true,
            'message' => 'Data mahasiswa berhasil ditambahkan.',
            'errors'   => [],
            'data'     => $data,
            'id'       => $id,
        ];
    }
}
```

Validasi (NIM wajib diisi, NIM harus angka, NIM tidak boleh duplikat, email wajib valid, program studi wajib tersedia, angkatan 4 digit, status harus salah satu dari aktif/cuti/lulus) seluruhnya ditempatkan pada method validate() di Service, bukan di Controller:

```
private function validate(array $data, ?string $exceptNim = null): array
{
    $errors = [];
```

```

        if ($data['nim'] === '') {
            $errors['nim'] = 'NIM wajib diisi';
        } elseif (!ctype_digit($data['nim'])) {
            $errors['nim'] = 'NIM harus berupa angka';
        } elseif ($this->repo->existsByNim($data['nim'], $exceptNim)) {
            $errors['nim'] = 'NIM sudah terdaftar';
        }

        if ($data['email'] === '') {
            $errors['email'] = 'Email wajib diisi';
        } elseif (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            $errors['email'] = 'Format email tidak valid';
        }

        if ($data['prodi_id'] <= 0) {
            $errors['prodi_id'] = 'Program studi wajib dipilih';
        } elseif (!$this->prodiRepo->existsById($data['prodi_id'])) {
            $errors['prodi_id'] = 'Program studi tidak tersedia';
        }

        if (!preg_match('/^\d{4}$/', $data['angkatan'])) {
            $errors['angkatan'] = 'Angkatan harus terdiri dari 4 angka';
        }

        if (!in_array($data['status'], self::STATUS_VALID, true)) {
            $errors['status'] = 'Status mahasiswa tidak valid';
        }

        return $errors;
    }
}

```

b. MahasiswaController menjadi tipis

MahasiswaController tidak lagi berisi validasi maupun query database. Controller hanya memanggil \$this->service dan menentukan response (tampilkan view, redirect, atau flash message).

```

class MahasiswaController extends BaseController
{
    public function __construct(MahasiswaService $service) {}

    public function store(): void
    {
        $result = $this->service->create($_POST);
    }
}

```

```

        if ($result['success']) {
            $this->flash('success', $result['message']);
            $this->redirect('/mahasiswa');
        }

        // Gagal: tampilkan flash (sekali tampil) dan kembalikan ke form
        // beserta pesan error per field dan isian sebelumnya.
        $this->flash('danger', $result['message']);

        $this->view(
            'mahasiswa/create',
            [
                'prodiList' => $this->service->prodiOptions(),
                'errors' => $result['errors'],
                'old' => $result['data'],
            ],
            'mahasiswa'
        );
    }

```

c. Flash Message dengan Session

Method flash() pada BaseController menyimpan pesan ke Session. Pesan ditampilkan satu kali oleh partials/flash.php, lalu langsung dihapus dari Session dengan unset().

```

// app/Core/BaseController.php
protected function flash(string $type, string $text): void
{
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }

    $_SESSION['flash_message'] = [
        'type' => $type,
        'text' => $text,
    ];
}

// app/Views/partials/flash.php
<?php if (!empty($_SESSION['flash_message'])): ?>
    <div class="alert alert-<?=  
htmlspecialchars($_SESSION['flash_message']['type']) ?> alert-dismissible fade  
show" role="alert">
        <?=  
htmlspecialchars($_SESSION['flash_message']['text']) ?>
    </div>
    <?php unset($_SESSION['flash_message']); ?>
<?php endif; ?>

```

3. Perbandingan Acara 12 dan Acara 13

Tabel 1. Perbandingan struktur kode sebelum dan sesudah penerapan Service Layer

Aspek	Acara 12	Acara 13
Validasi data mahasiswa	Dilakukan langsung di Controller	Dipindahkan ke MahasiswaService::validate()
Tugas Controller	Menerima request, validasi, dan memanggil Repository	Hanya menerima request, memanggil Service, lalu menentukan response
Pengecekan NIM duplikat	Dicek manual di Controller sebelum simpan	Dicek di dalam Service melalui Repository::existsByNim()
Notifikasi ke pengguna	Belum konsisten (tampil langsung di halaman yang sama)	Flash Message melalui Session, tampil sekali lalu dihapus otomatis
Penanganan hasil proses	Return value bercampur dengan tampilan	Service mengembalikan array terstruktur (success, message, errors, data)

4. Pemetaan dengan Tugas Praktikum

Tabel 2. Pemetaan tugas pada modul dengan implementasi

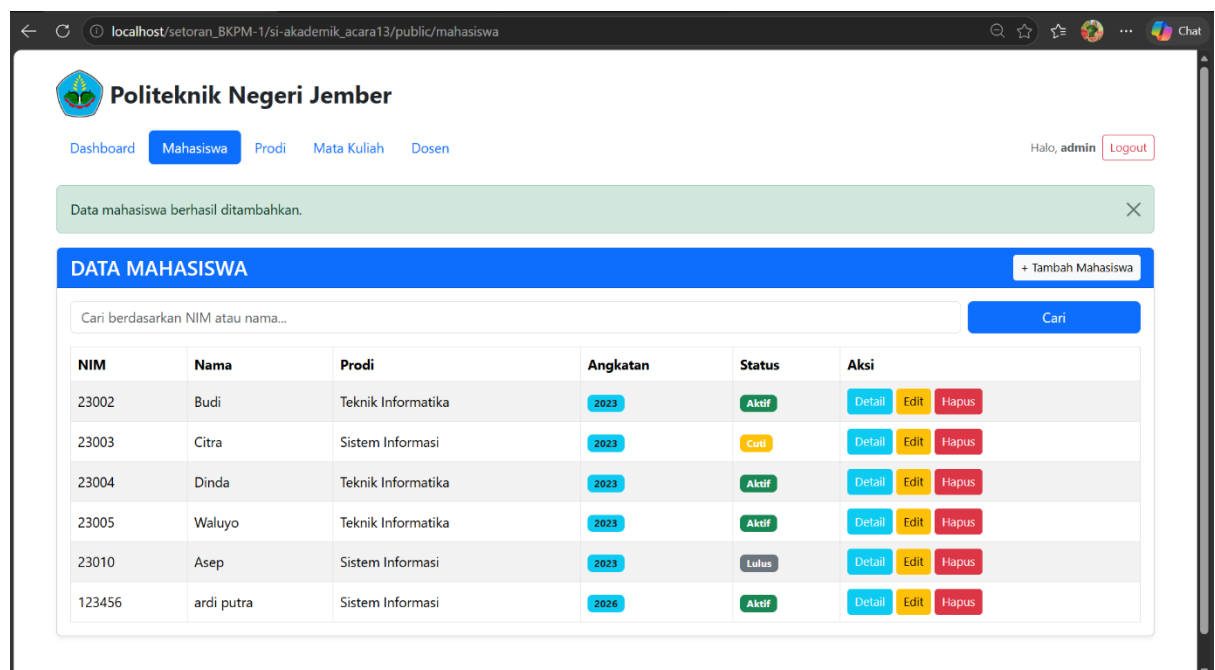
N o	Tugas pada Modul	Implementasi	File
1	Membuat class MahasiswaService	Constructor injection MahasiswaRepository & ProdiRepository	app/Services/MahasiswaService.php
2	Inject Repository melalui constructor	private MahasiswaRepository \$repo, ProdiRepository \$prodiRepo	app/Services/MahasiswaService.php
3	Pindahkan validasi dari Controller ke Service	Method validate()	app/Services/MahasiswaService.php
4	Buat method create()	create(array \$input): array	app/Services/MahasiswaService.php
5	Buat method update()	update(string \$oldNim, array \$input): array	app/Services/MahasiswaService.php
6	Controller tidak validasi/query langsung	Controller hanya memanggil \$this->service	app/Controllers/MahasiswaController.php

N o	Tugas pada Modul	Implementasi	File
7	Flash Message dengan Session	Method flash() pada BaseController	app/Core/BaseController.php
8	Hapus flash setelah ditampilkan	unset(\$_SESSION['flash_message'])	app/Views/partials/flash.php
9	Uji seluruh proses	Diuji pada browser (lihat bagian 5)	-

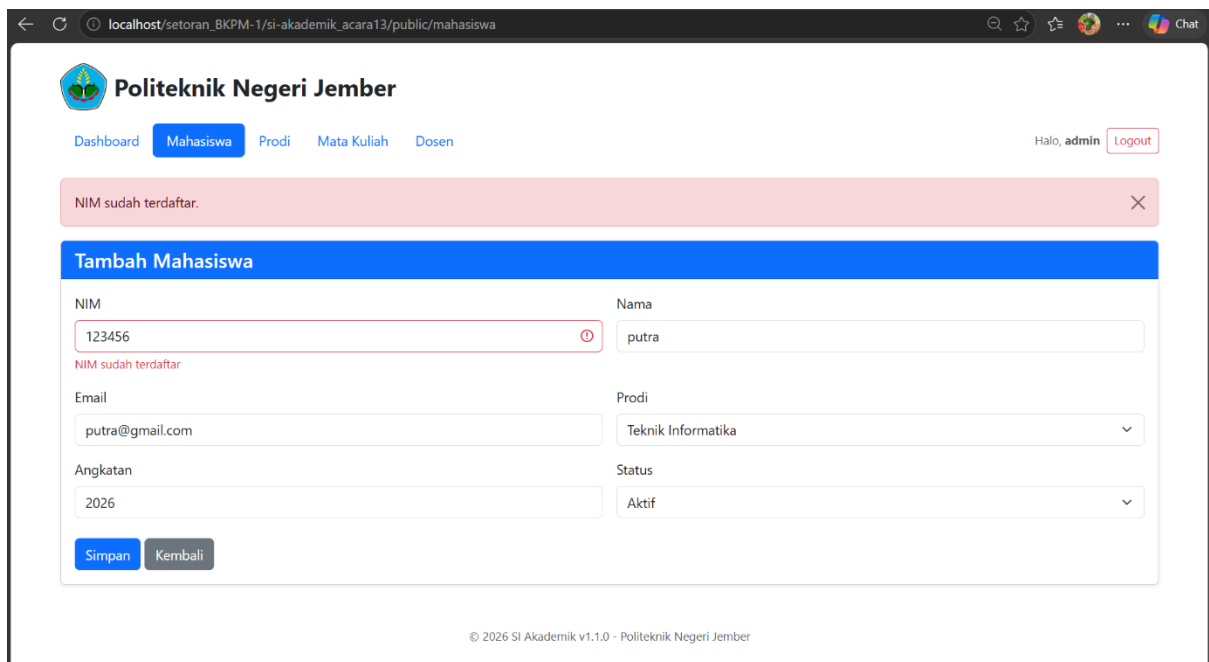
5. Pengujian Aplikasi

Pengujian dilakukan melalui browser untuk memastikan Service Layer dan Flash Message berjalan sesuai skenario berikut:

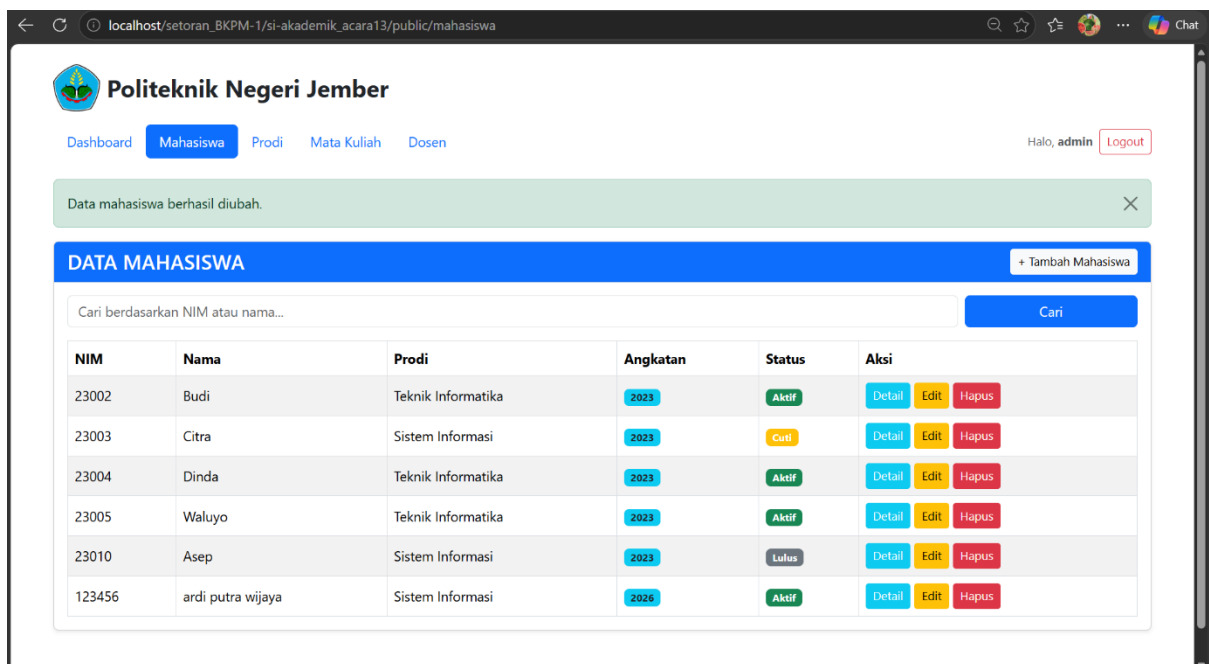
- Menambahkan mahasiswa dengan data valid → muncul flash message sukses “Data mahasiswa berhasil ditambahkan.”
- Menambahkan mahasiswa dengan NIM yang sudah terdaftar → muncul flash message “NIM sudah terdaftar.” dan data form tidak hilang (old input).
- Mengubah data mahasiswa → muncul flash message “Data mahasiswa berhasil diubah.”
- Menambahkan mahasiswa dengan data tidak valid (misalnya format email salah) → muncul flash message “Data gagal disimpan. Periksa kembali isian Anda.” beserta pesan error pada field terkait.
- Me-refresh halaman setelah flash message tampil → pesan tidak muncul lagi (sudah terhapus dari Session).



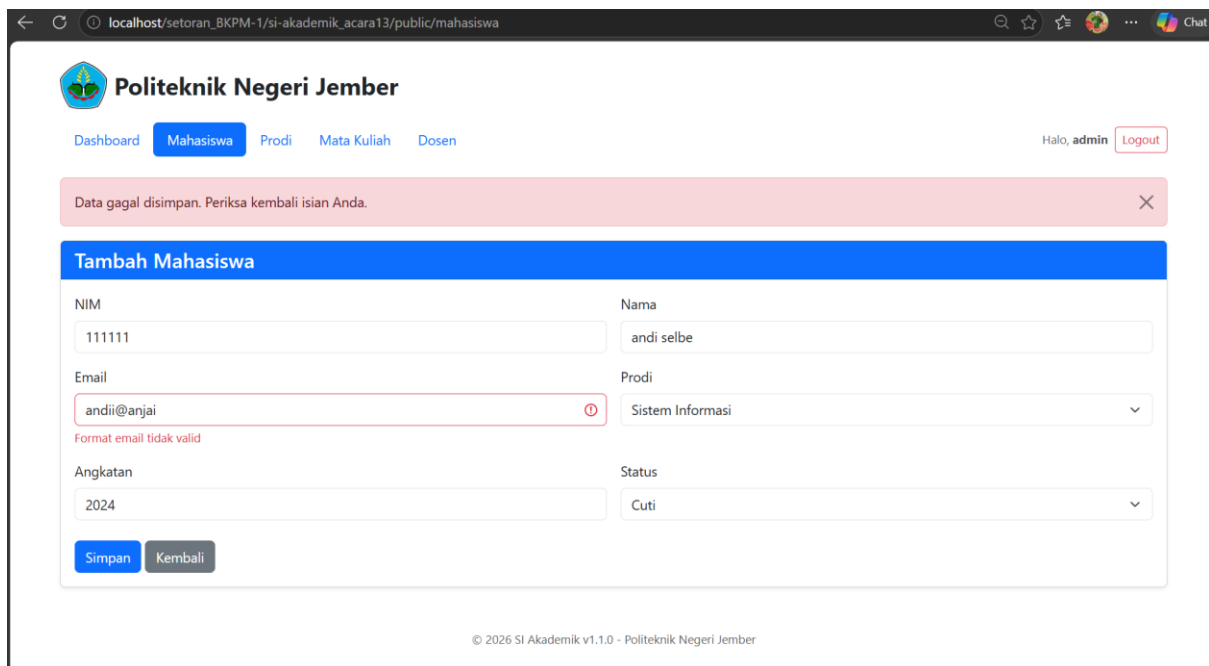
Gambar 1. Tambah data mahasiswa – flash message sukses



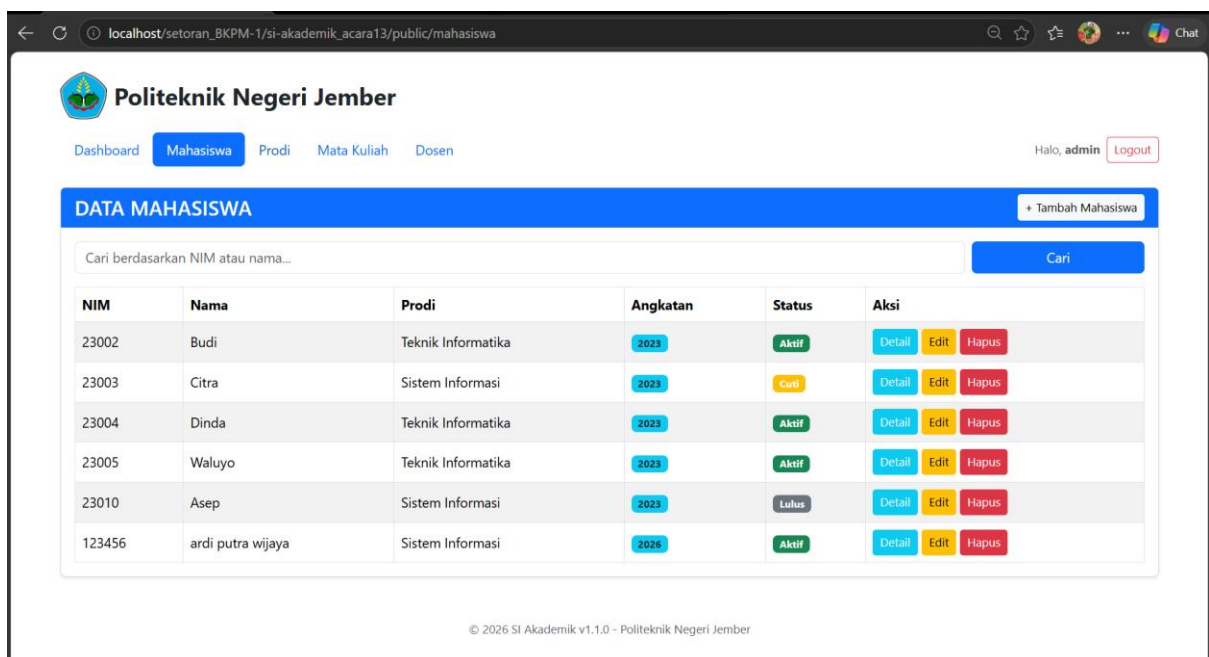
Gambar 2. Tambah data dengan NIM duplikat – flash message gagal



Gambar 3. Ubah data mahasiswa – flash message sukses



Gambar 4. Tambah data dengan email tidak valid – flash message gagal disimpan



Gambar 5. Halaman di-refresh setelah flash message tampil – pesan sudah hilang

6. Pembahasan

Penerapan Service Layer memisahkan logika bisnis dari Controller. MahasiswaController tidak lagi berisi validasi maupun query database, sehingga kode menjadi lebih sederhana, lebih mudah dibaca, dan lebih mudah diuji secara terpisah dari urusan HTTP request/response.

Flash Message dengan Session memberikan cara yang konsisten untuk menyampaikan hasil proses kepada pengguna tanpa perlu mengirim ulang data melalui query string. Karena pesan

dihapus setelah ditampilkan, pengguna tidak akan melihat pesan yang sama berulang kali saat melakukan refresh halaman.

KESIMPULAN

1. MahasiswaService berhasil memindahkan seluruh proses validasi dan logika bisnis keluar dari Controller, menjadikan Controller hanya bertugas menerima request dan menentukan response.
2. Dependency Injection pada constructor MahasiswaService (MahasiswaRepository dan ProdiRepository) membuat Service tidak perlu mengetahui detail query SQL, sepenuhnya diserahkan ke Repository.
3. Flash Message berbasis Session berhasil memberikan notifikasi satu kali kepada pengguna dan terhapus otomatis setelah ditampilkan, baik untuk pesan sukses maupun gagal (termasuk kasus NIM duplikat).
4. Fitur tambah dan ubah data mahasiswa tetap berjalan dengan baik setelah proses refactoring ke Service Layer, dan pesan error dari database (PDOException) tetap tertangani tanpa menghentikan aplikasi.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.